S3 Bucket and IAM Access Control Experiment

1 Objective

The objective of this experiment is to understand practical access control in Amazon S3 by using an existing IAM user to create and manage S3 resources. The experiment covers custom IAM permissions, S3 bucket creation, object upload, bucket policies, object-level ACLs, and access to S3 objects through browser URLs.

The experiment also demonstrates the difference between an IAM policy, an S3 bucket policy, and an object ACL.

2 Description

The experiment was performed using an existing IAM user. A new IAM user was not created for this work. A custom IAM policy was attached to the existing user to provide the S3 permissions required for the experiment.

The workflow was:

Existing IAM User → Custom S3 Permissions → Create Bucket → Upload Test Objects → Bucket Policy → Object ACL → S3 Object Access

Two intentional test objects were used:

1. An RTO application screenshot.
2. A Google Colab screenshot.

The RTO screenshot used in this document has been sanitized so that personal information is not reproduced.

3 Topology

The topology diagram supplied for this experiment is used directly below. It is stored in the project root as `S3.drawio.png`.

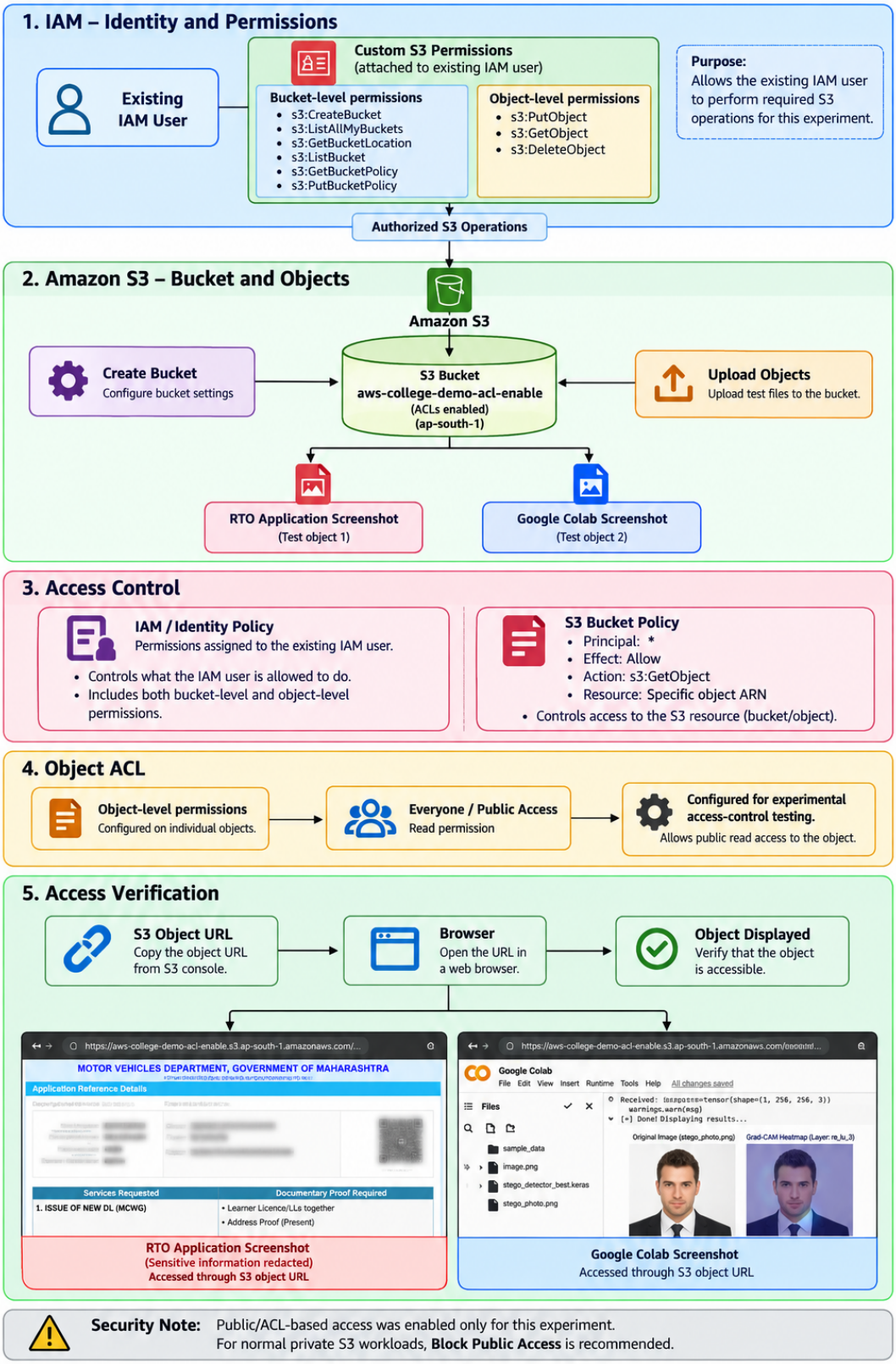


Figure 1: S3 bucket and IAM access-control topology for the experiment.

4 IAM User and Custom Permissions

The experiment used the existing IAM user. The user was given a custom policy containing the S3 and policy-validation actions required for the practical.

4.1 Policy creation and assignment

The policy creation screen shows a custom policy being prepared for the IAM user.

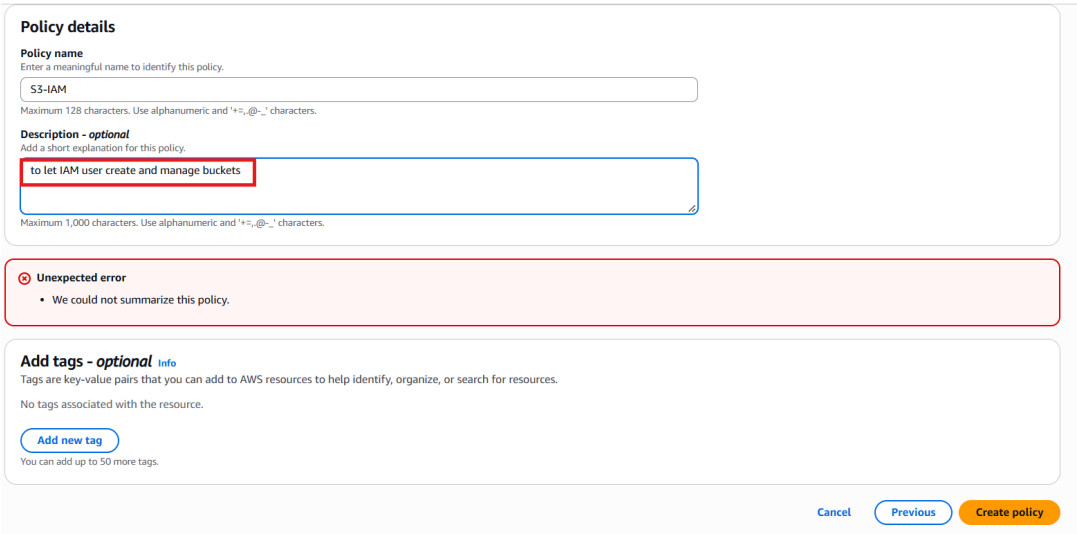


Figure 2: Creating the custom IAM policy for S3 operations.

The policy was then located from the IAM policy list.

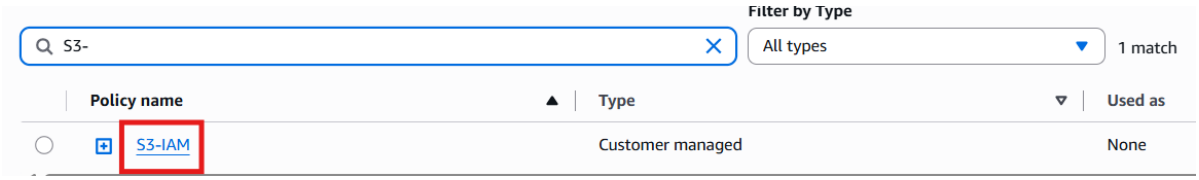


Figure 3: Locating the custom policy in the IAM policy list.

The IAM user was reviewed with the custom policy listed under its permissions.

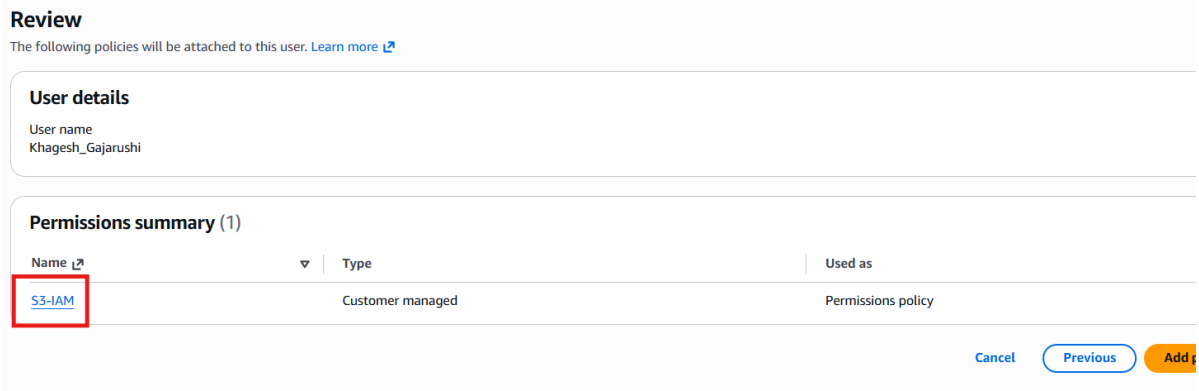


Figure 4: Review of the IAM user's custom S3 policy.

The user's permissions page confirms that the custom policy is attached to the existing IAM user.

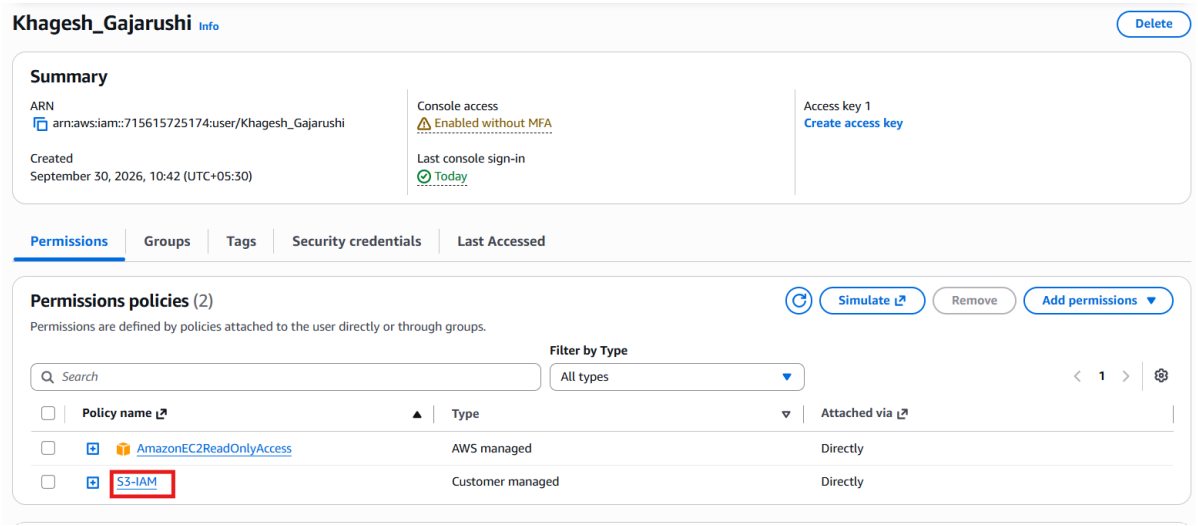


Figure 5: Existing IAM user with the custom policy attached.

4.2 Complete IAM policy

The IAM user was given the following policy. The complete action list is included because these permissions were part of the actual experiment.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "VisualEditor0",
            "Effect": "Allow",
            "Action": [
                "s3:GetBucketPublicAccessBlock",
                "s3:PutBucketPublicAccessBlock",
                "s3:PutBucketOwnershipControls",
                "s3:CreateBucket",
                "s3:ListBucket",
                "s3:GetBucketAcl",
                "s3:GetBucketPolicy",
                "s3:PutObject",
                "s3:GetObjectAcl",
                "s3:GetObject",
                "s3:ListAllMyBuckets",
                "access-analyzer:ValidatePolicy",
                "s3:PutBucketPolicy",
                "s3:DeleteObject",
                "s3:GetBucketLocation",
                "s3:PutObjectAcl"
            ],
            "Resource": "*"
        }
    ]
}
```

4.3 Permission summary

Permission	Purpose
s3:CreateBucket	Create an S3 bucket.
s3:ListAllMyBuckets	List the S3 buckets available to the user.
s3:GetBucketLocation	Retrieve a bucket's location.
s3:ListBucket	List objects in a bucket.
s3:GetBucketAcl	Read the bucket ACL.
s3:GetBucketPublicAccessBlock	Read the bucket's public-access-block settings.
s3:PutBucketPublicAccessBlock	Change the bucket's public-access-block settings.
s3:PutBucketOwnershipControls	Configure bucket object-ownership controls.
s3:GetBucketPolicy	Read the bucket policy.
s3:PutBucketPolicy	Add or edit the bucket policy.
s3:PutObject	Upload objects.
s3:GetObject	Read or access objects.
s3:DeleteObject	Delete objects.
s3:GetObjectAcl	Read an object's ACL.
s3:PutObjectAcl	Change an object's ACL.
access-analyzer:ValidatePolicy	Validate an IAM/resource policy with IAM Access Analyzer.

Bucket-level permissions operate on the bucket resource itself, such as listing objects, reading bucket settings, or changing the bucket policy. Object-level permissions operate on individual objects, such as uploading, reading, deleting, or changing an object's ACL.

5 Opening Amazon S3

After the IAM permissions were assigned, Amazon S3 was opened from the AWS console.

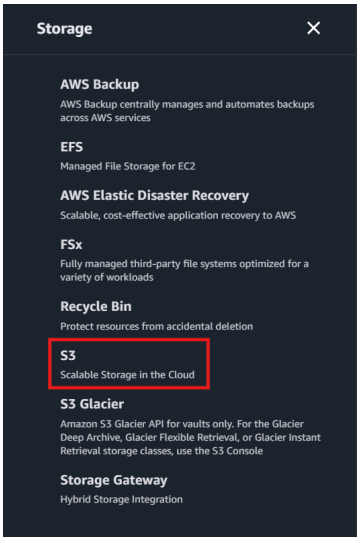


Figure 6: Selecting Amazon S3 from the AWS console.

The S3 bucket page was then opened to begin the bucket configuration.

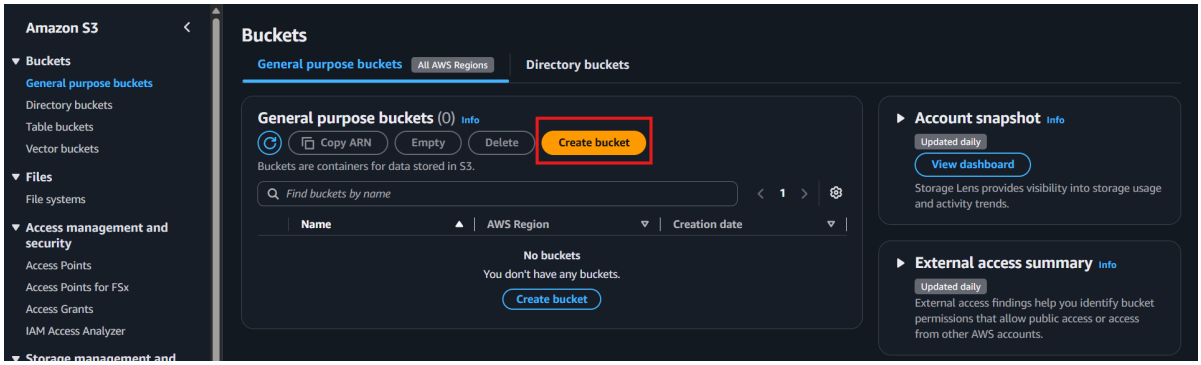


Figure 7: Amazon S3 Buckets page.

6 S3 Bucket Creation

The *Create bucket* option was selected and the bucket configuration was entered.

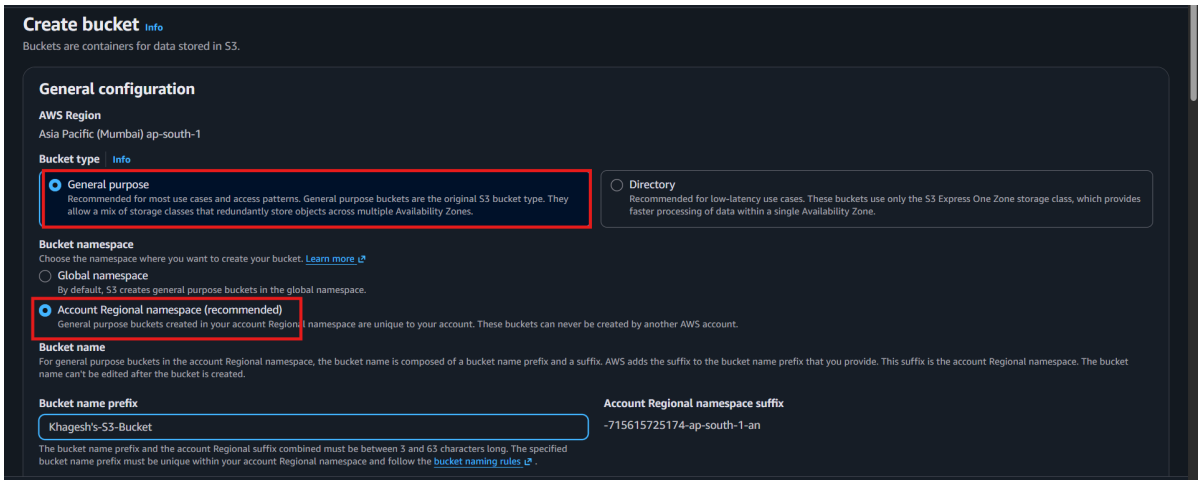


Figure 8: Initial S3 bucket configuration.

For this experiment, ACL support was intentionally enabled because object-level ACLs were going to be tested later. The bucket-owner-preferred ownership option was selected.

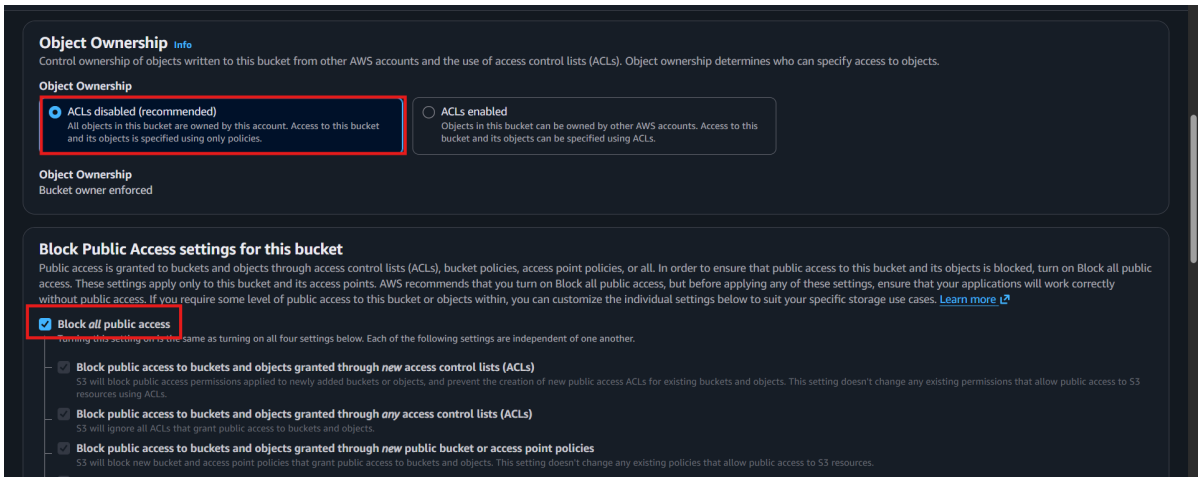


Figure 9: Object Ownership configuration with ACLs enabled for the experiment.

The remaining bucket settings were reviewed before creation.

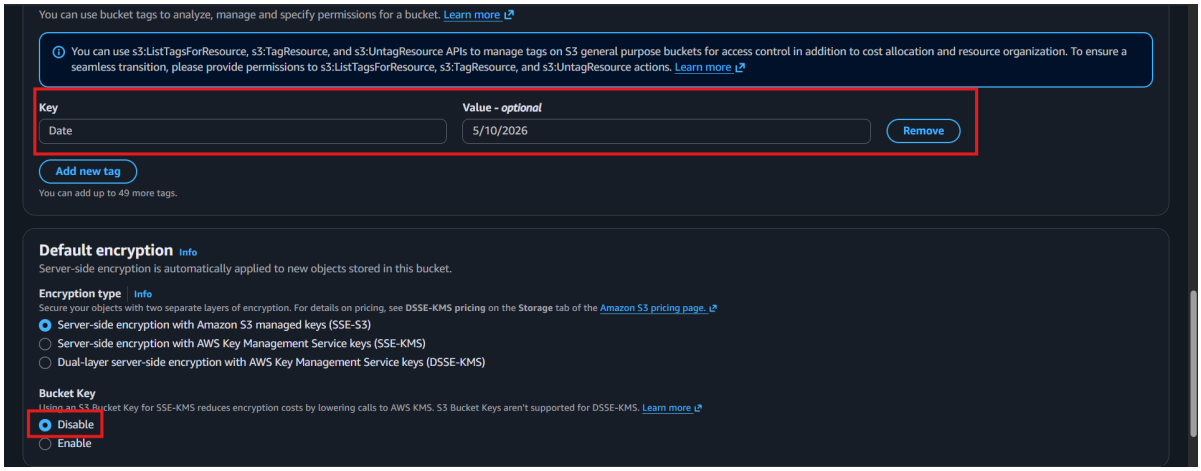


Figure 10: Reviewing the bucket settings before creation.

The bucket was successfully created.

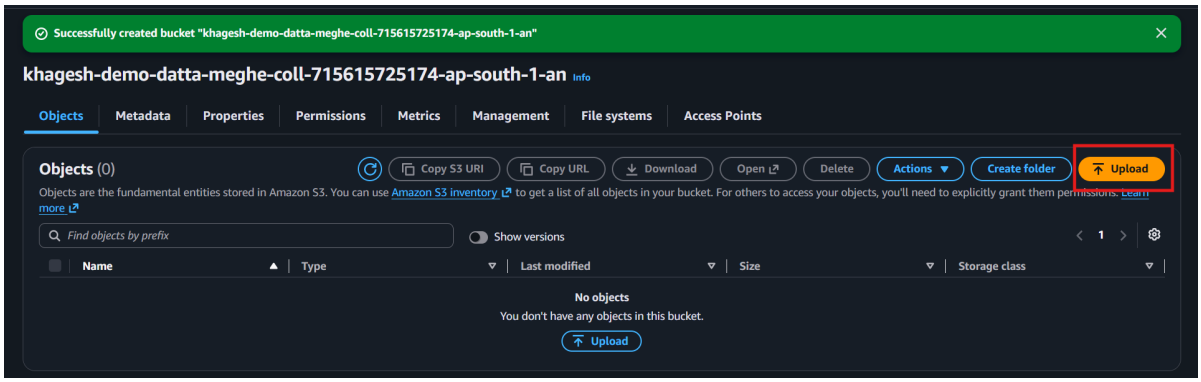


Figure 11: Successful S3 bucket creation.

Note: AWS recommends disabling ACLs for normal modern S3 workloads unless a specific use case requires object-level ACL control. ACLs were enabled here specifically because they were part of the practical experiment.

7 Uploading Test Objects

The first upload was prepared from the S3 bucket's upload interface.

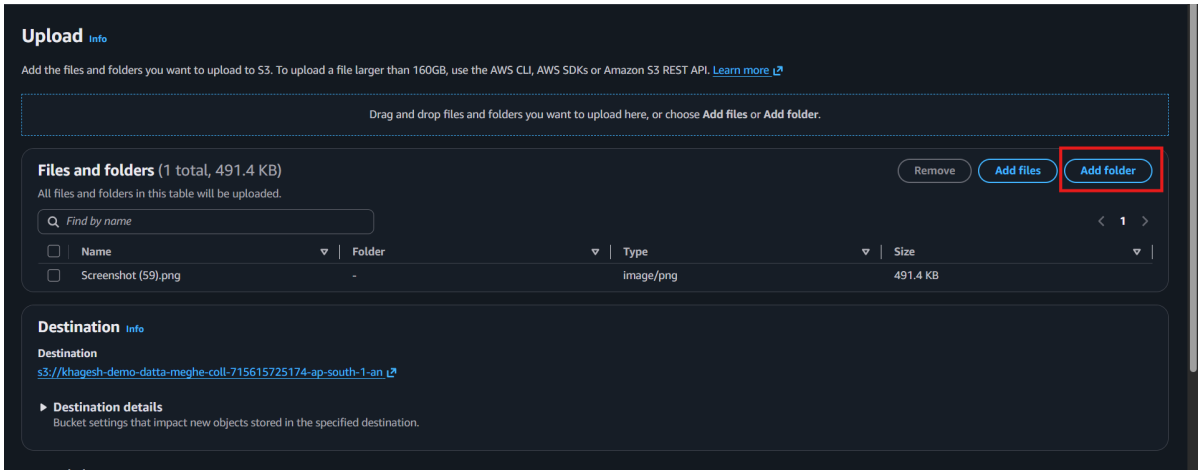


Figure 12: S3 upload screen with the test object selected.

The upload completed successfully.

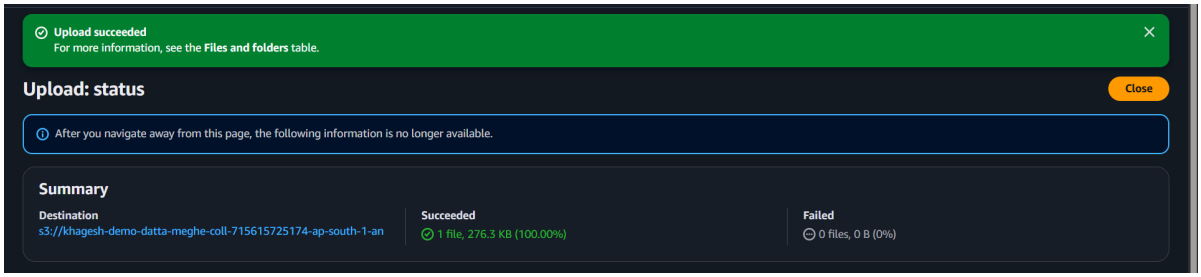


Figure 13: Successful upload confirmation.

The uploaded object was then visible in the bucket.

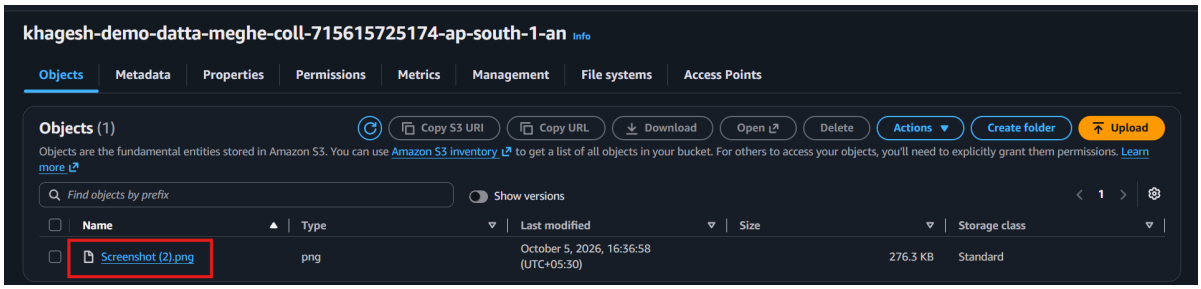


Figure 14: Uploaded object listed in the S3 bucket.

The object properties were opened to inspect its details.

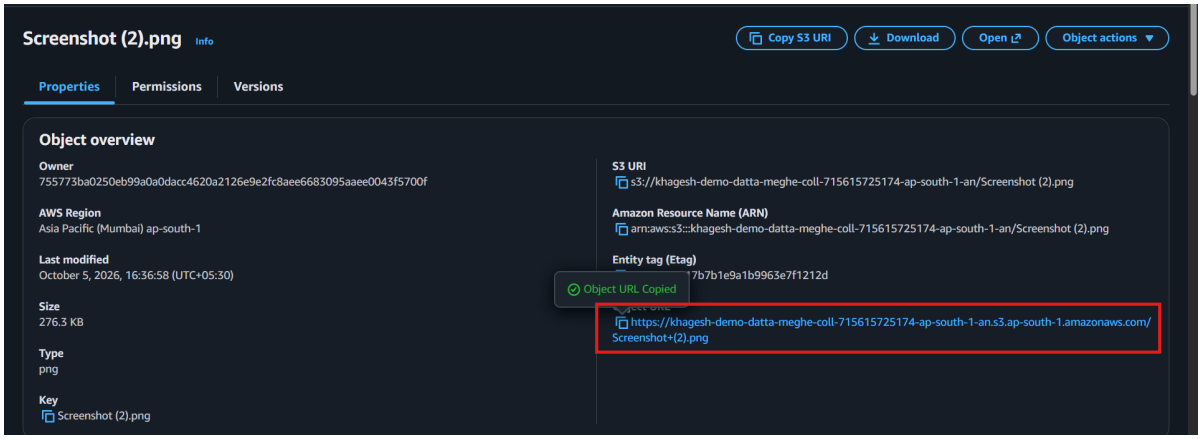


Figure 15: Object properties for the uploaded S3 object.

8 Initial Object URL Access

An initial attempt was made to access the object directly through its S3 object URL.

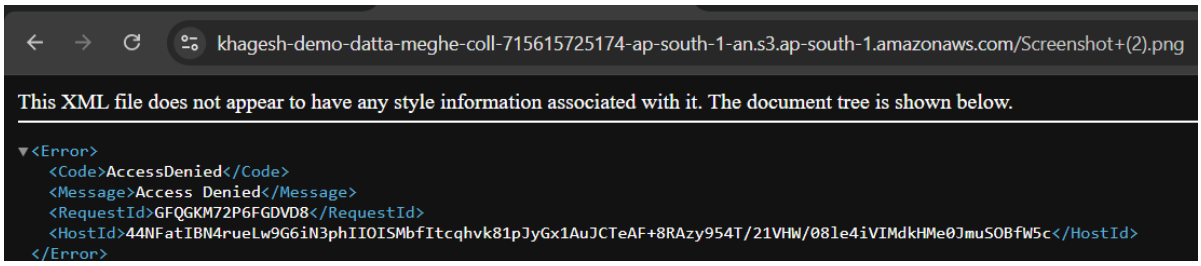


Figure 16: Initial direct object URL attempt returning an AccessDenied response.

This demonstrates that having an object stored in S3 does not by itself mean that an unauthenticated browser request can read it. Access depends on the permissions and resource configuration applied to the object.

A presigned URL was then generated from the S3 object actions.

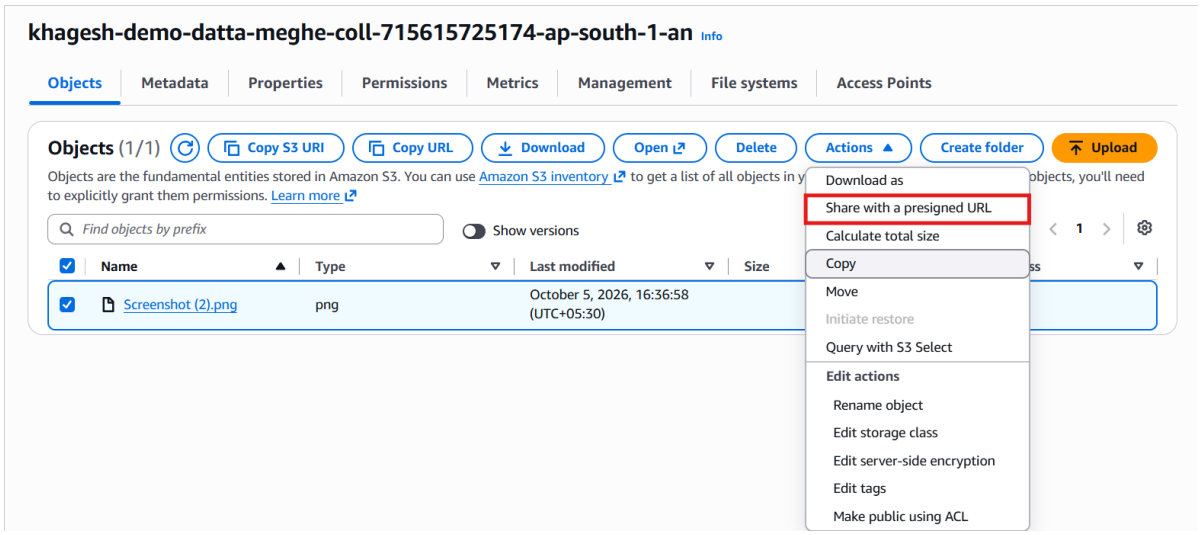


Figure 17: Object actions menu used to generate a presigned URL.

The presigned URL configuration was displayed.

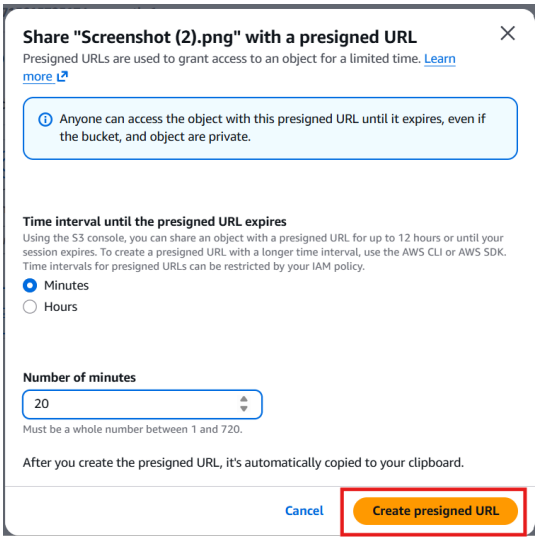


Figure 18: Presigned URL configuration for the S3 object.

The presigned URL was created and copied.

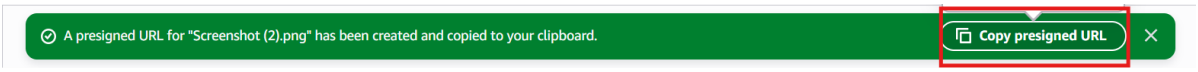


Figure 19: Successful creation and copying of the presigned URL.

The object could then be displayed through the generated URL.

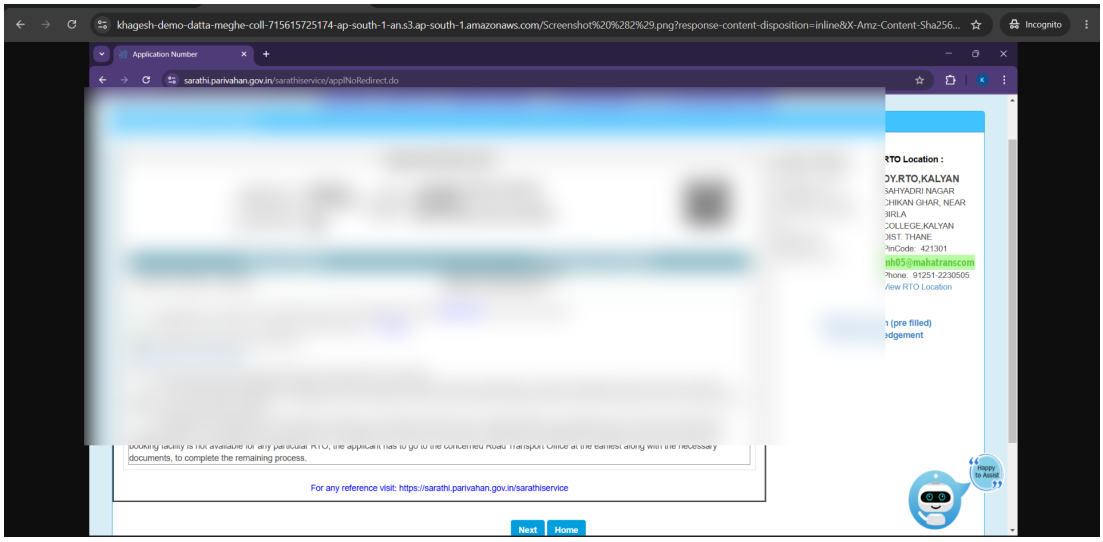


Figure 20: RTO application screenshot accessed through the S3 URL after applying the configured access permissions. Sensitive personal information has been redacted.

9 Bucket Policy

The next part of the experiment configured a resource-based S3 bucket policy using the AWS Policy Generator.

The policy generator was configured for an `Allow` statement with:

- Principal: `*`

- Action: `s3:GetObject`
- Resource: a specific S3 object ARN

The resource points to a specific object rather than every object in the bucket.

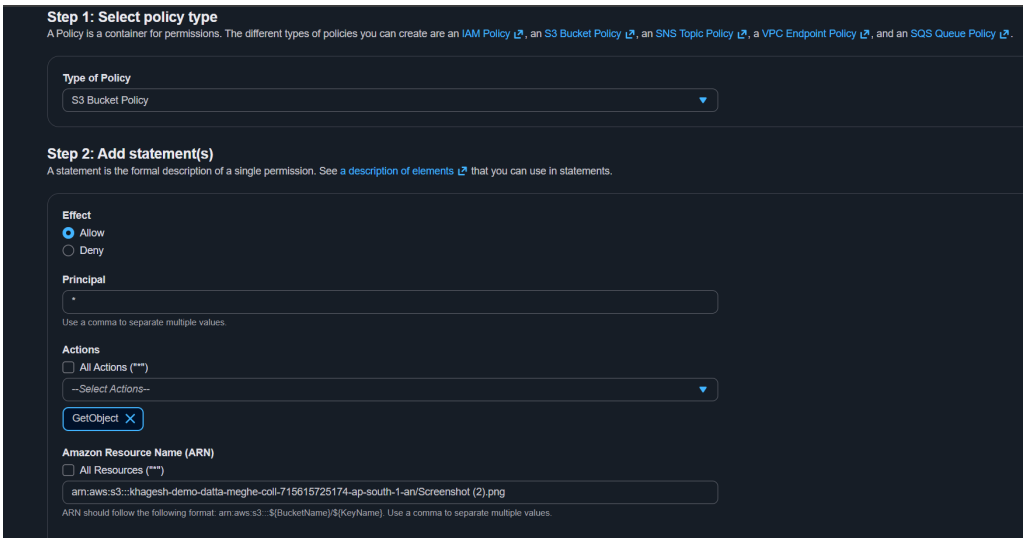


Figure 21: AWS Policy Generator configured for an S3 object-access statement.

The statement was added and the policy was ready to be generated.

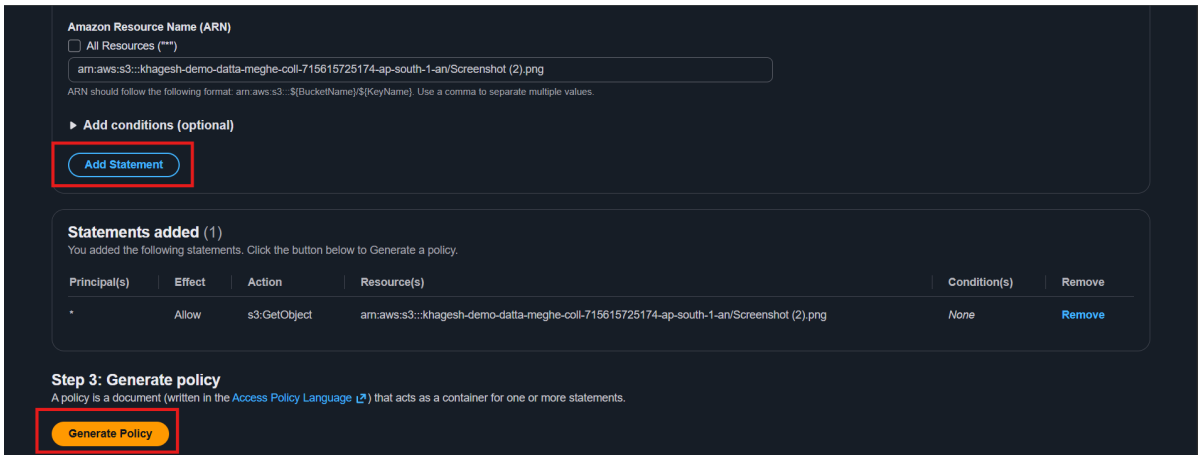


Figure 22: AWS Policy Generator showing the completed statement before generating the policy.

The generated JSON policy was displayed.

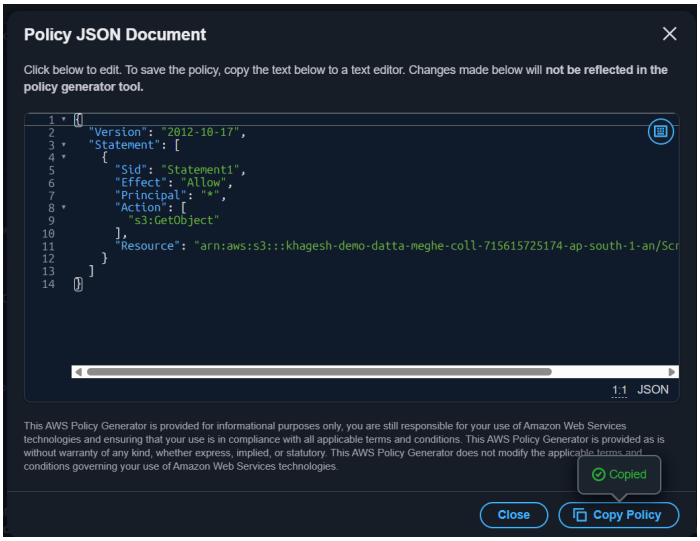


Figure 23: Generated bucket-policy JSON.

The S3 bucket Permissions page was then opened.

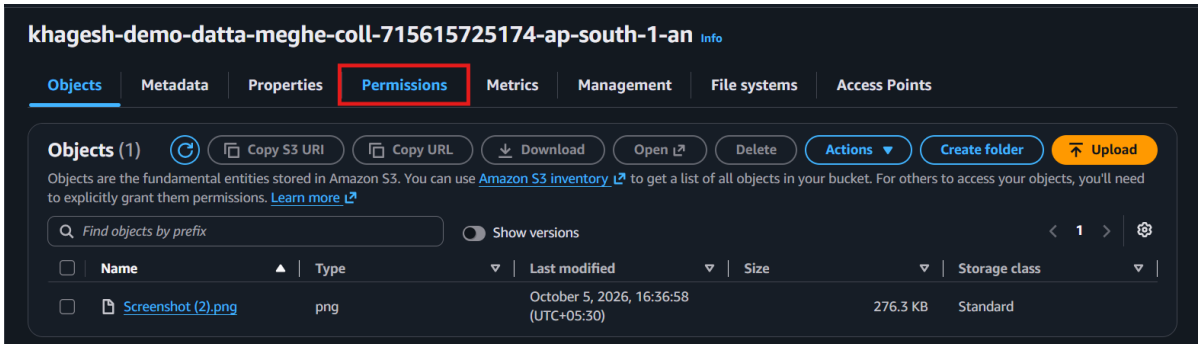


Figure 24: S3 bucket Permissions page.

The Block Public Access settings were reviewed before applying the bucket policy.

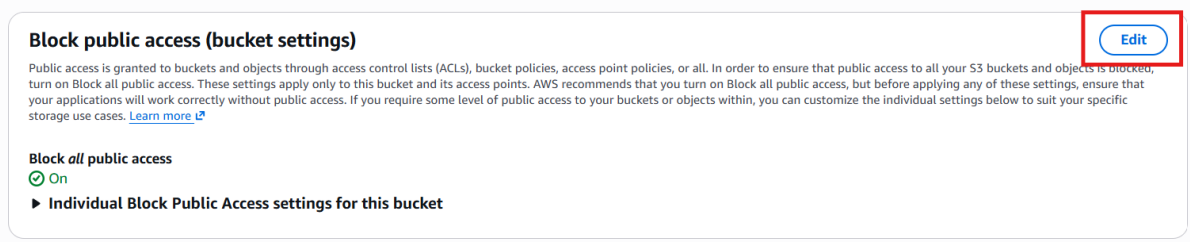


Figure 25: S3 bucket Block Public Access settings.

The individual public-access controls were edited for the experimental configuration.

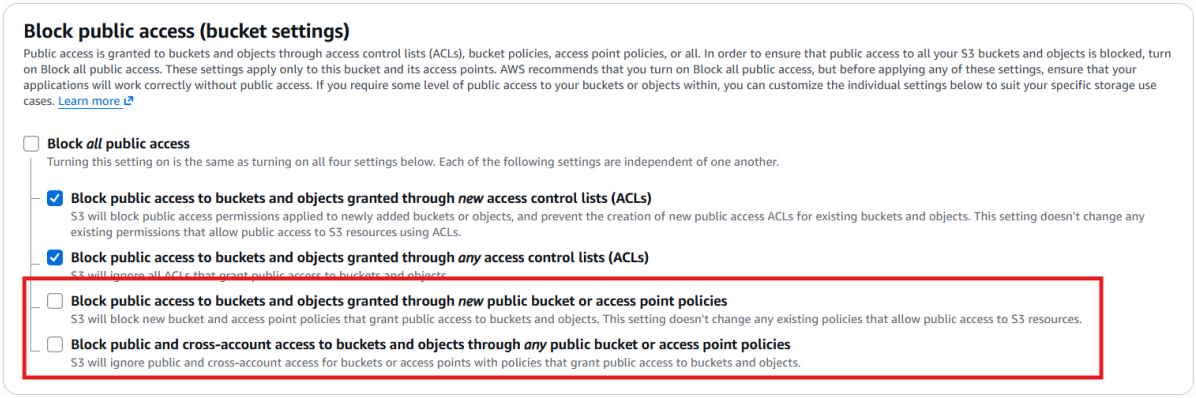


Figure 26: Editing the bucket's Block Public Access configuration.

The Bucket Policy section was opened for editing.

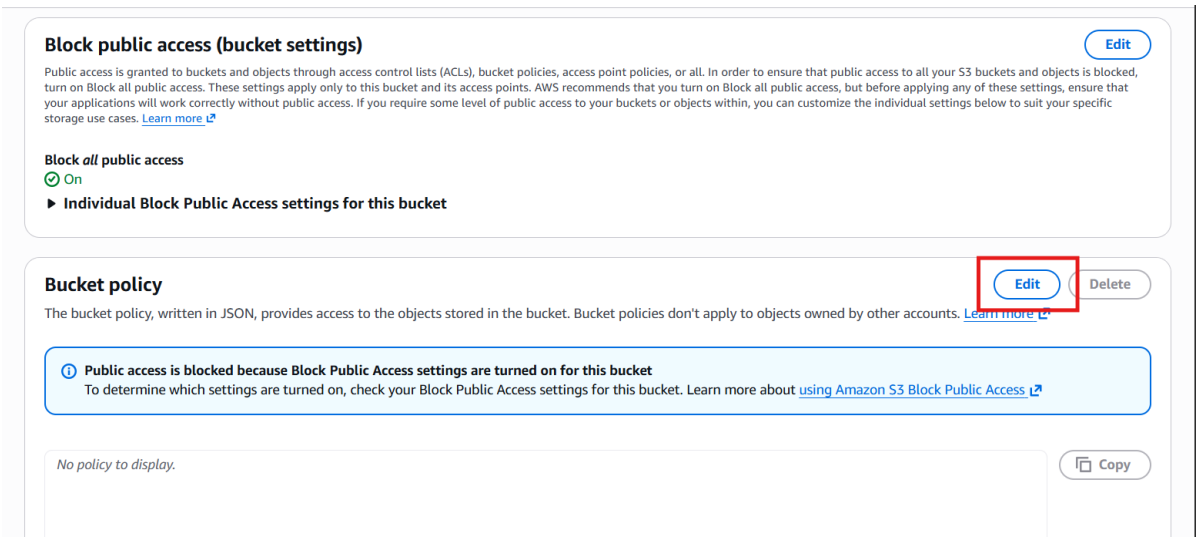


Figure 27: Opening the bucket policy editor.

The generated policy JSON was entered into the bucket policy editor.

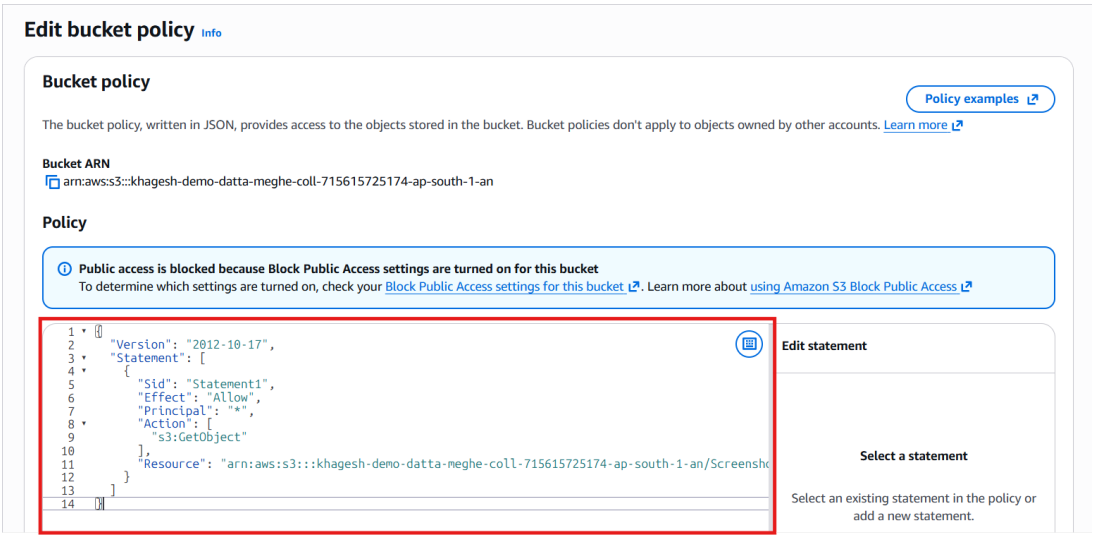


Figure 28: Bucket policy editor containing the generated policy.

The policy update completed successfully.



Figure 29: Successful bucket-policy update confirmation.

9.1 Bucket ARN and Object ARN

A bucket ARN identifies the bucket itself:

```
arn:aws:s3:::bucket-name
```

An object ARN identifies an individual object within that bucket:

```
arn:aws:s3:::bucket-name/object-key
```

The experiment used the object ARN form for the `s3:GetObject` resource. Therefore, the demonstrated statement was limited to the specified object and did not automatically grant `GetObject` access to every object in the bucket.

10 Object ACL Configuration

ACL configuration was performed separately from the bucket policy.

The bucket-creation configuration shows that ACLs were enabled and that *Bucket owner preferred* was selected.

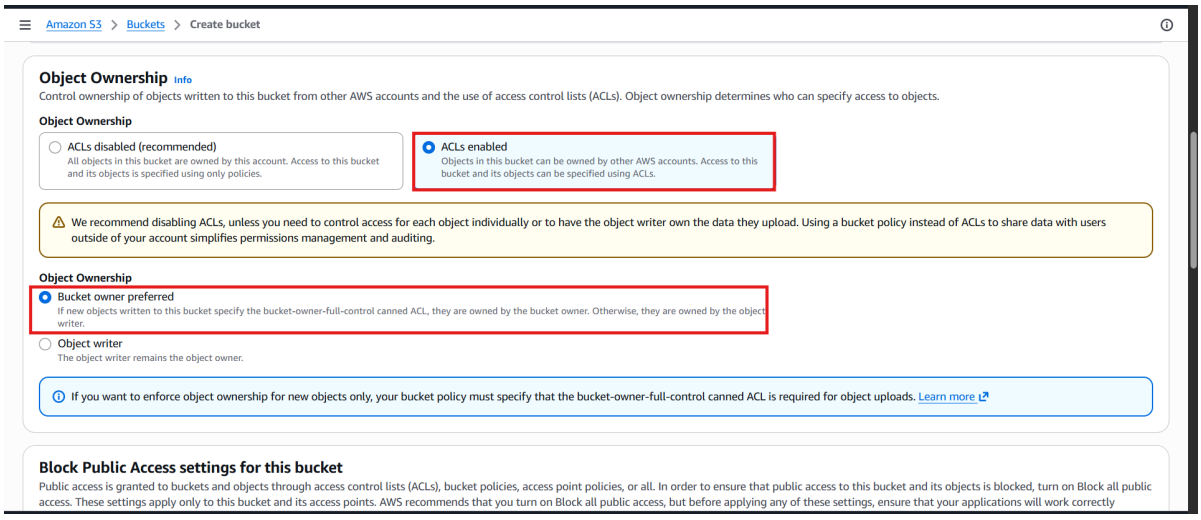


Figure 30: ACL-enabled Object Ownership configuration used for the ACL experiment.

The experimental ACL bucket was successfully created.

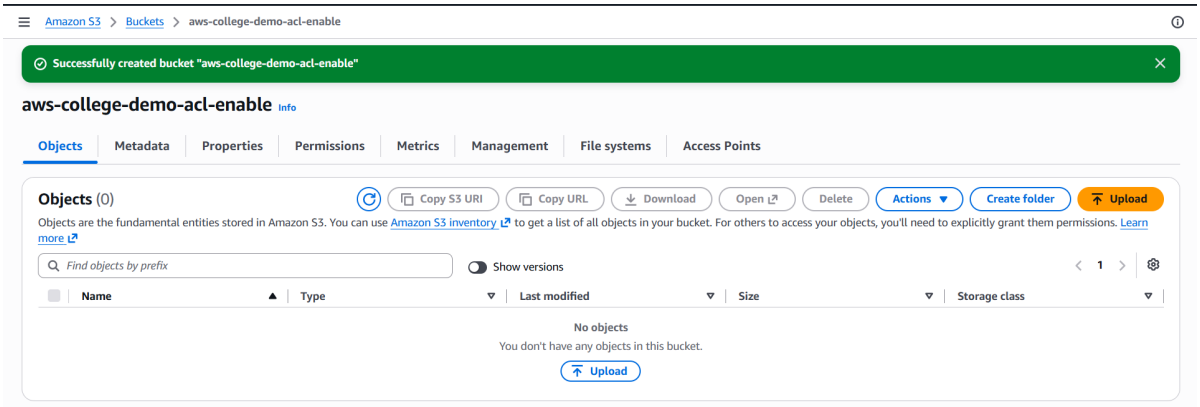


Figure 31: Successful creation of the ACL-enabled experimental bucket.

A test object was uploaded successfully.

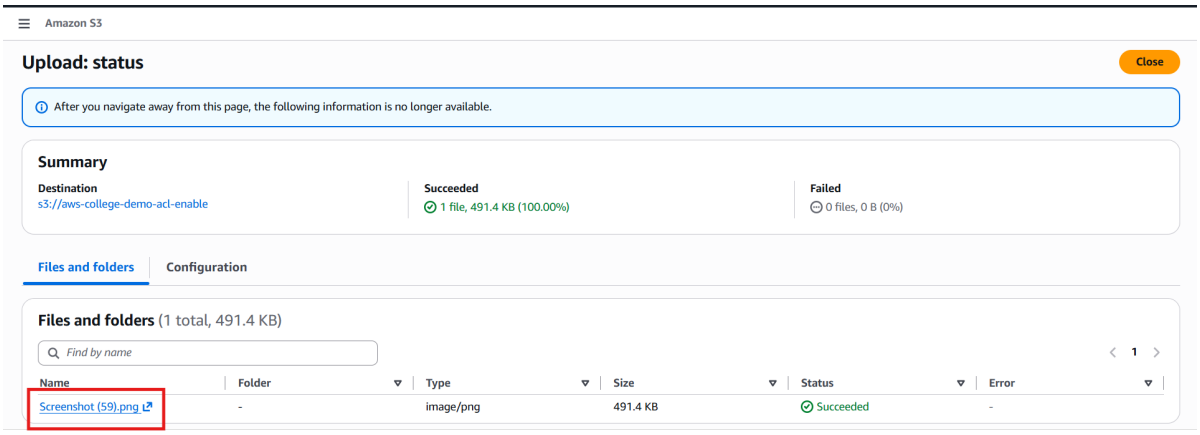


Figure 32: Successful upload of the test object to the ACL-enabled bucket.

The bucket's Block Public Access configuration was then edited so that the ACL-based public-access settings could be tested.

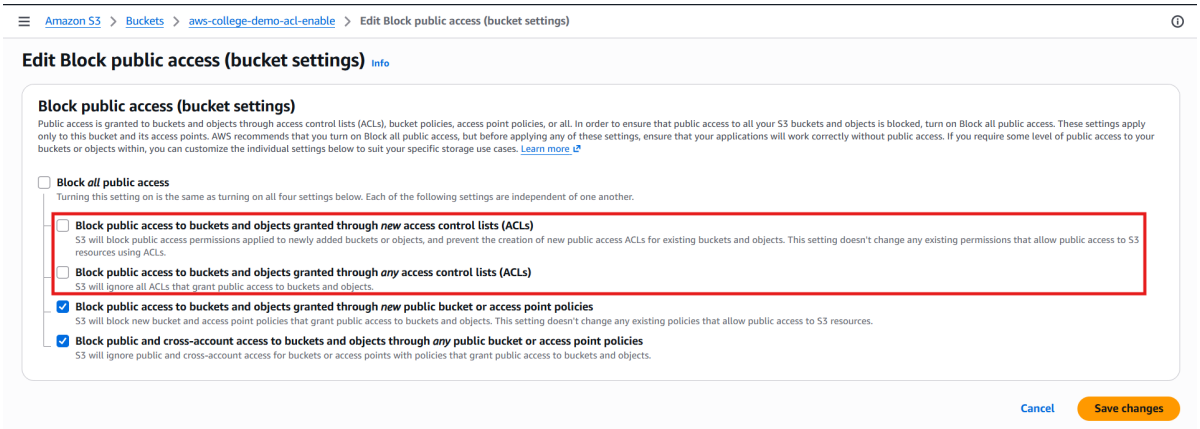


Figure 33: Editing Block Public Access settings for the ACL experiment.

The object Permissions page was opened and the Access Control List section was selected.

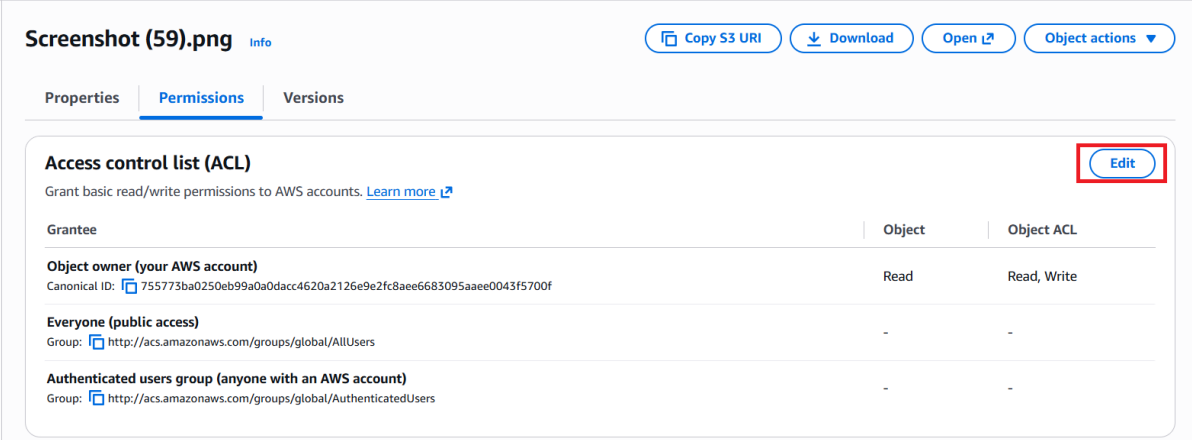


Figure 34: Object Permissions page showing the Access Control List section.

The object ACL was edited to grant the *Everyone (public access)* grantee read permission. The AWS warning was acknowledged before saving the change.

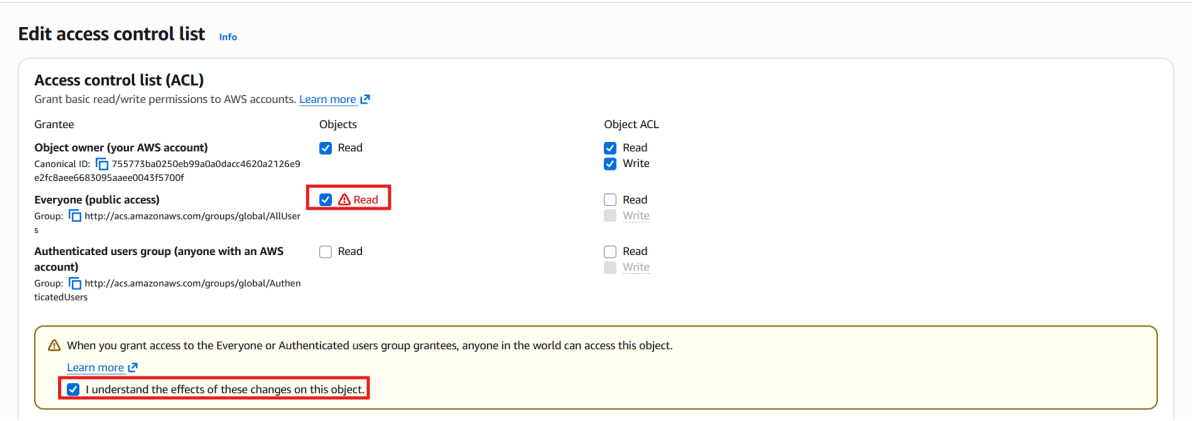


Figure 35: Object ACL configured with public read permission and the AWS warning acknowledged.

11 Second Test Object and Access Verification

The second intentional test object was a Google Colab screenshot. It was used to verify that the S3 access configuration could display another uploaded object through the S3 object URL.

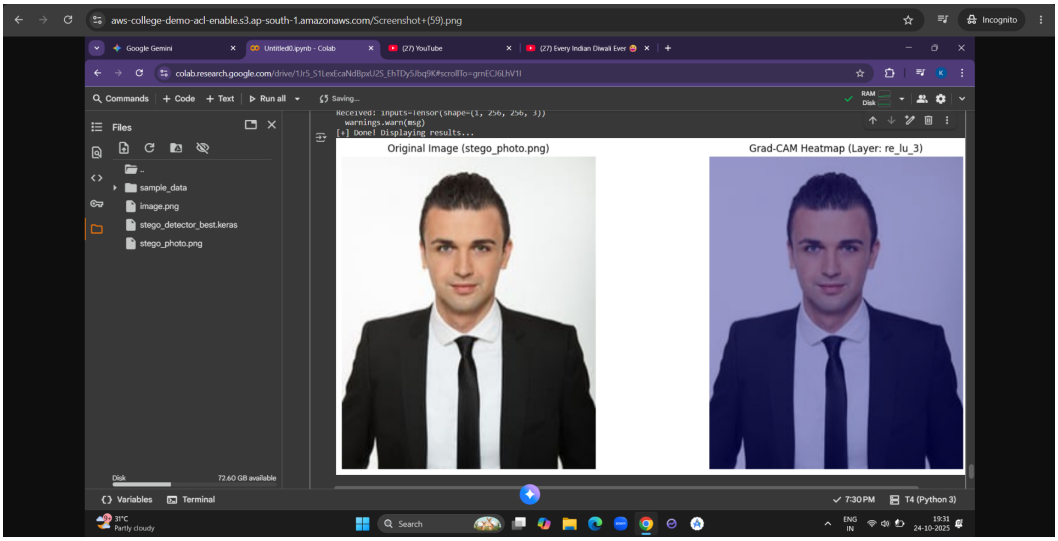


Figure 36: Google Colab screenshot displayed through the S3 object URL.

12 IAM Policy vs Bucket Policy vs Object ACL

The three access-control mechanisms used or discussed in the experiment have different roles:

Mechanism	Role in the experiment
IAM Policy	Attached to the IAM user. It determines which AWS actions the user is allowed to perform, including S3 bucket and object operations.
Bucket Policy	A resource-based policy attached to an S3 bucket. The experiment used it to allow `s3:GetObject` for a specified object resource.
Object ACL	An object-level access-control mechanism. The experiment used it to grant public read access to an individual object.

The IAM policy answers the question *"What can this IAM identity do?"*. The bucket policy and object ACL are resource-side controls that determine access to the S3 resource.

13 Security Note

Public access was enabled only for experimental purposes. In normal private S3 deployments, Block Public Access should remain enabled unless public access is explicitly required.

The object ACL screen also shows AWS's warning that granting access to the Everyone or Authenticated Users groups can expose the object to users outside the account. Public object access should therefore be treated as a deliberate configuration rather than a default setting.

14 Observations

- The existing IAM user was able to work with S3 using the assigned custom permissions.
- Bucket-level permissions and object-level permissions serve different purposes.

- The experiment used ACLs intentionally so that object-level permissions could be demonstrated.
- A bucket ARN identifies the bucket, while an object ARN identifies a particular object.
- The bucket-policy statement demonstrated in the experiment used `s3:GetObject` on a specific object ARN.
- An object URL can return `AccessDenied` when the required access is not configured.
- A presigned URL provides temporary access to an object, while a public ACL or public bucket policy changes how the resource can be accessed.
- Public access was used only for the experiment and should not be treated as the normal configuration for private S3 data.

15 Conclusion

This experiment provided a practical view of Amazon S3 access control. An existing IAM user was given custom S3 permissions, an S3 bucket was created, test objects were uploaded, and access was tested through S3 URLs.

The experiment also demonstrated three separate layers of access control: the IAM policy assigned to the user, the bucket policy attached to an S3 bucket, and the ACL applied to an individual object. Understanding the difference between these layers is important when configuring S3 permissions and controlling public or private access to stored objects.